

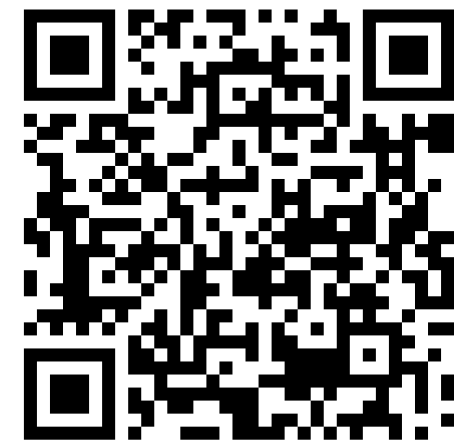
ARCHITECTURE MICROSERVICES

SOCIAL MEDIA BACKEND

PRÉSENTÉ PAR
AYA ANNABI & NOUR LAKHAL

SUPERVISÉ PAR
MME ASMA BAGHDADI

LIEN VERS LE PROJET



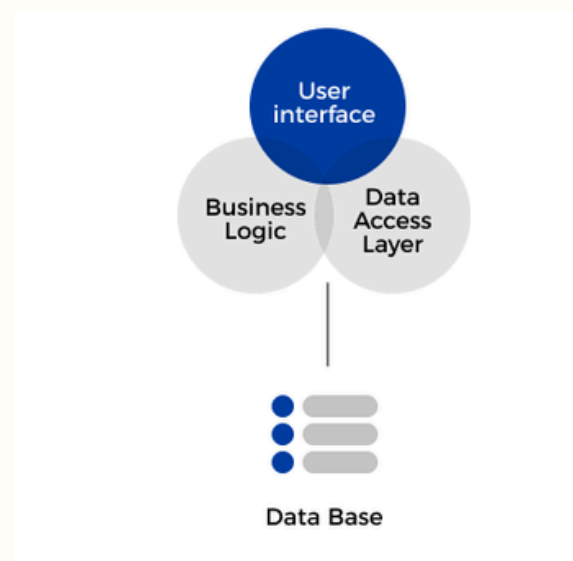
<https://github.com/EYAannabi/Tp-architecture-microservice.git>

- I. INTRODUCTION ET PROBLÉMATIQUE**
- II. LE PATTERN MICROSERVICES UTILISÉ**
- III. ARCHITECTURE & ORGANISATION DU PROJET**
- IV. ZOOM SUR L'API GATEWAY**
- V. PRÉSENTATION DES SERVICES MÉTIERS**
- VI. INTÉGRATION DE L'INTERFACE UTILISATEUR
(FRONTEND)**
- VII. SYNTHÈSE FINALE**

I. INTRODUCTION ET PROBLÉMATIQUE

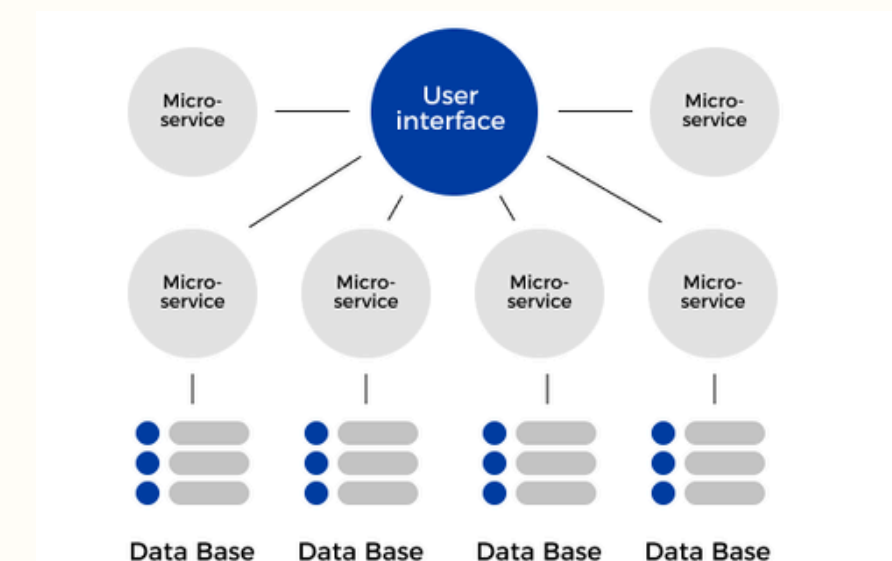
Pourquoi choisir les Microservices ?

LE DÉFI DU MONOLITHE



TRADITIONNELLEMENT, LES APPLICATIONS SONT CONSTRUITES COMME UNE UNITÉ UNIQUE (MONOLITHE). BIEN QUE SIMPLE AU DÉBUT, CETTE APPROCHE DEVIENT PROBLÉMATIQUE AVEC LA CROISSANCE D'UN RÉSEAU SOCIAL : UNE SEULE ERREUR DANS LE MODULE DES "STORIES" PEUT FAIRE TOMBER TOUT LE SYSTÈME, Y COMPRIS L'AUTHENTIFICATION

POURQUOI CHOISIR LES MICROSERVICES ?

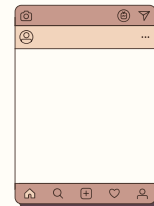


- **SCALABILITÉ CIBLÉE** : NOUS POUVONS ALLOUER PLUS DE RESSOURCES UNIQUEMENT AUX SERVICES LES PLUS SOLLICITÉS
- **ISOLATION DES PANNES** : SI LE SERVICE DE "STORIES" RENCONTRE UN BUG, LES UTILISATEURS PEUVENT TOUJOURS SE CONNECTER ET PUBLIER DES MESSAGES.
- **INDÉPENDANCE TECHNOLOGIQUE** : CHAQUE MICROSERVICE EST AUTONOME, PERMETTANT AUX ÉQUIPES DE TRAVAILLER EN PARALLÈLE.

II. LE PATTERN MICROSERVICES UTILISÉ

1. DÉCOMPOSITION PAR DOMAINE MÉTIER (DOMAIN-DRIVEN DESIGN)

POST SERVICE



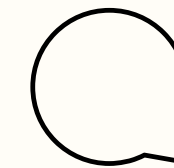
CYCLE DE VIE COMPLET
DES PUBLICATIONS

USER SERVICE



GESTION EXCLUSIVE DE
L'IDENTITE ET DES
(ABONNÉS)

COMMENT & STORY
SERVICES



GESTION DES
INTERACTIONS ET DU
CONTENU

2. LE PATTERN API GATEWAY (UNIQUE ENTRYPOINT)

ROUTAGE INTELLIGENT

Elle centralise les requêtes du port 3000 et les redirige vers les services internes

SÉCURITÉ CENTRALISÉE

Elle gère l'authentification JWT et le filtrage en un point unique. Si le token est invalide, la requête n'atteint même pas nos services métiers.

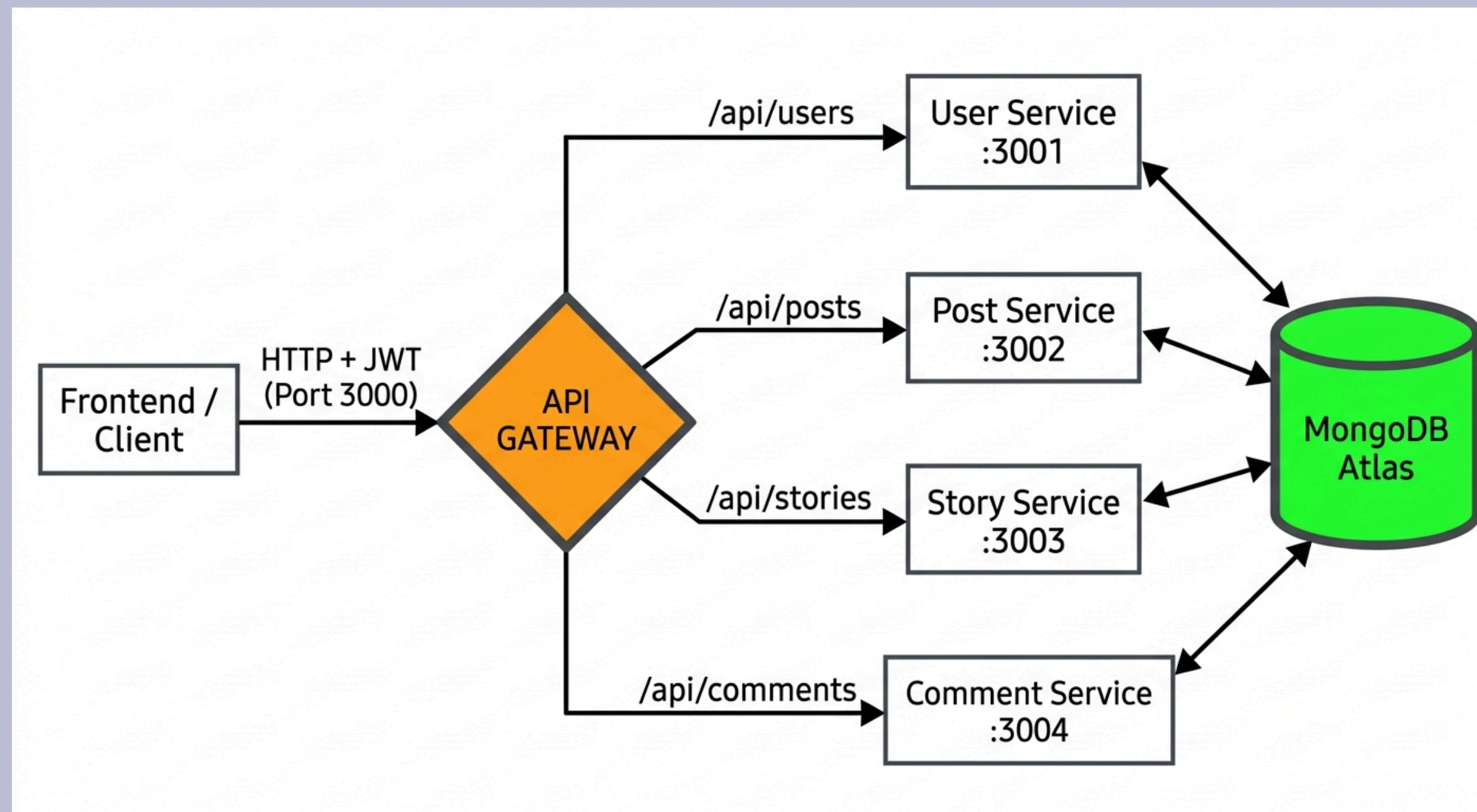
ABSTRACTION

Elle masque la complexité de l'infrastructure au monde extérieur, offrant une API cohérente et simplifiée au développeur Frontend.

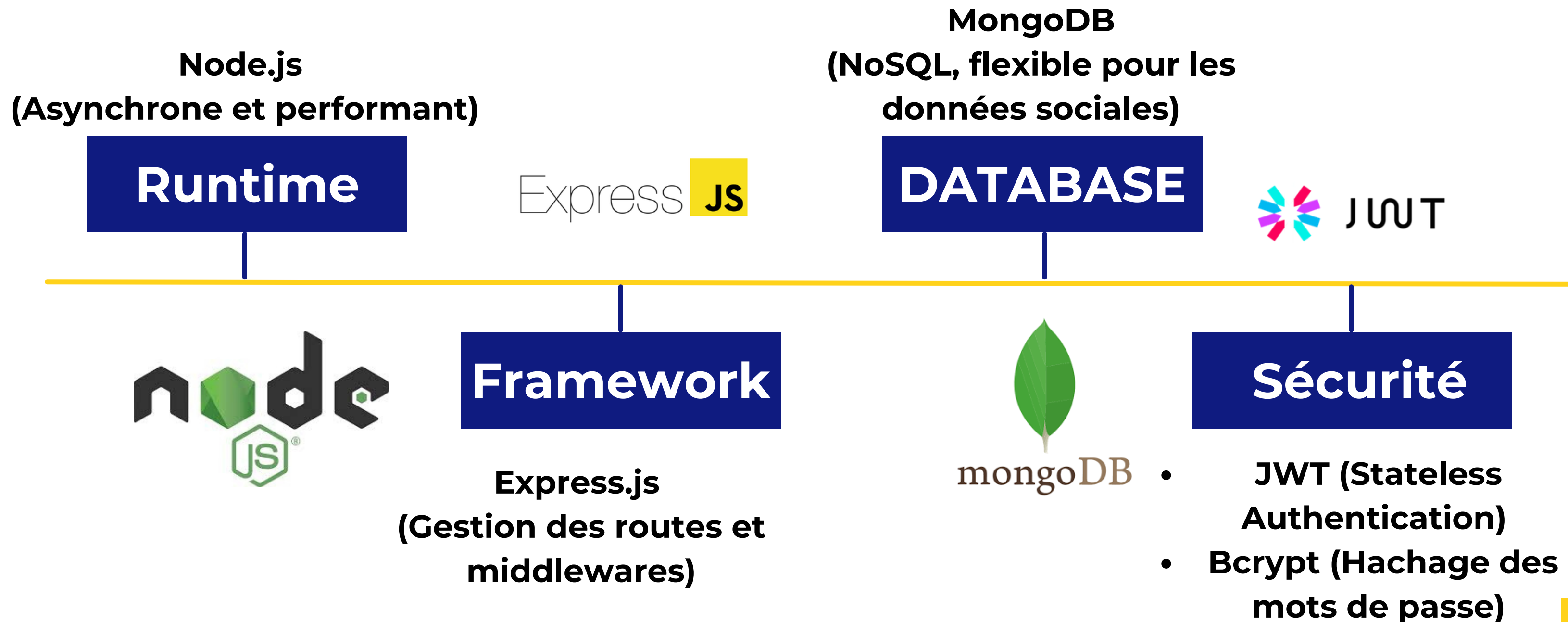
III. ARCHITECTURE & ORGANISATION DU PROJET

1. DIAGRAMME DE FLUX

Le système suit un flux linéaire et sécurisé :



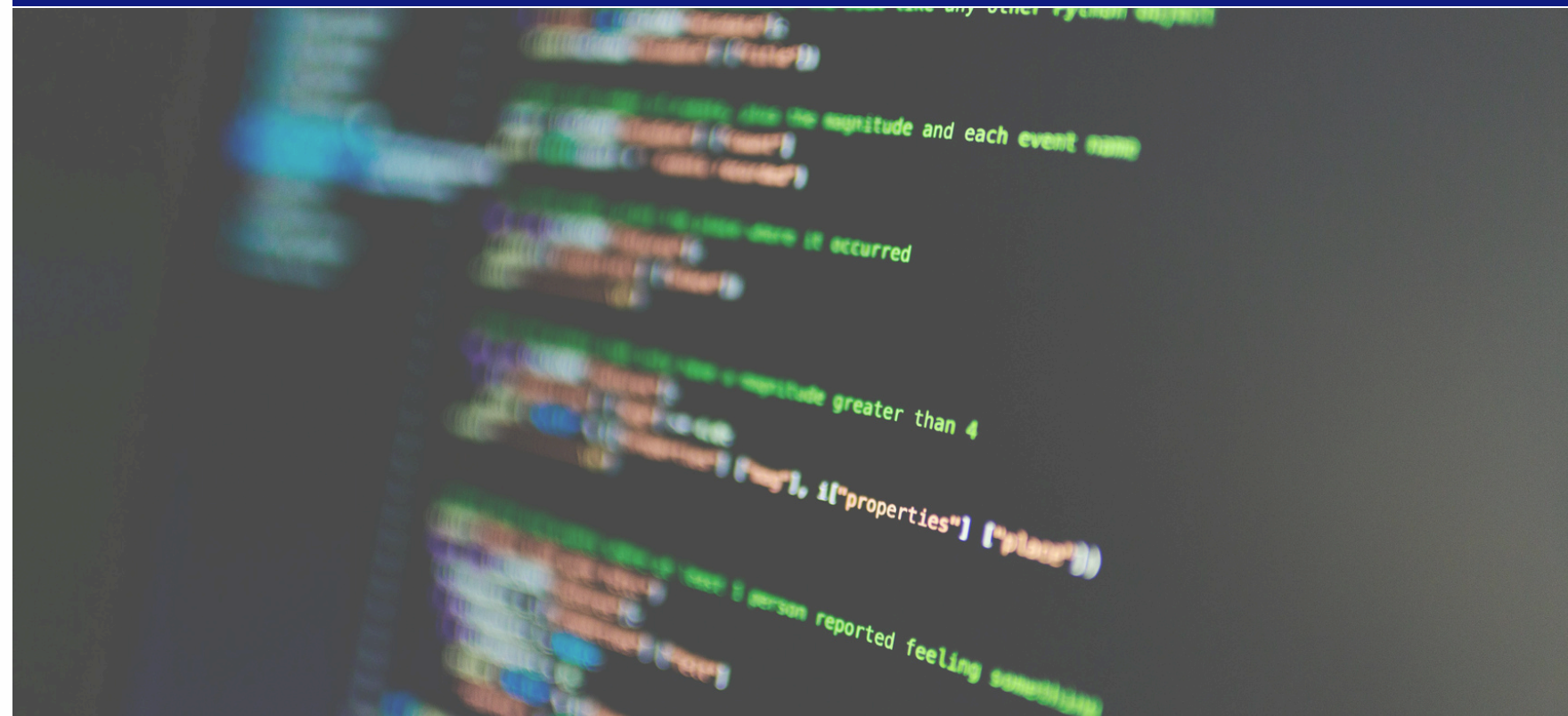
2. Stack Technique (L'écosystème)



3. Structure du Projet

Autonomie Totale :

Chaque dossier (user-service, post-service, etc.) possède son propre fichier package.json et son propre dossier node_modules.



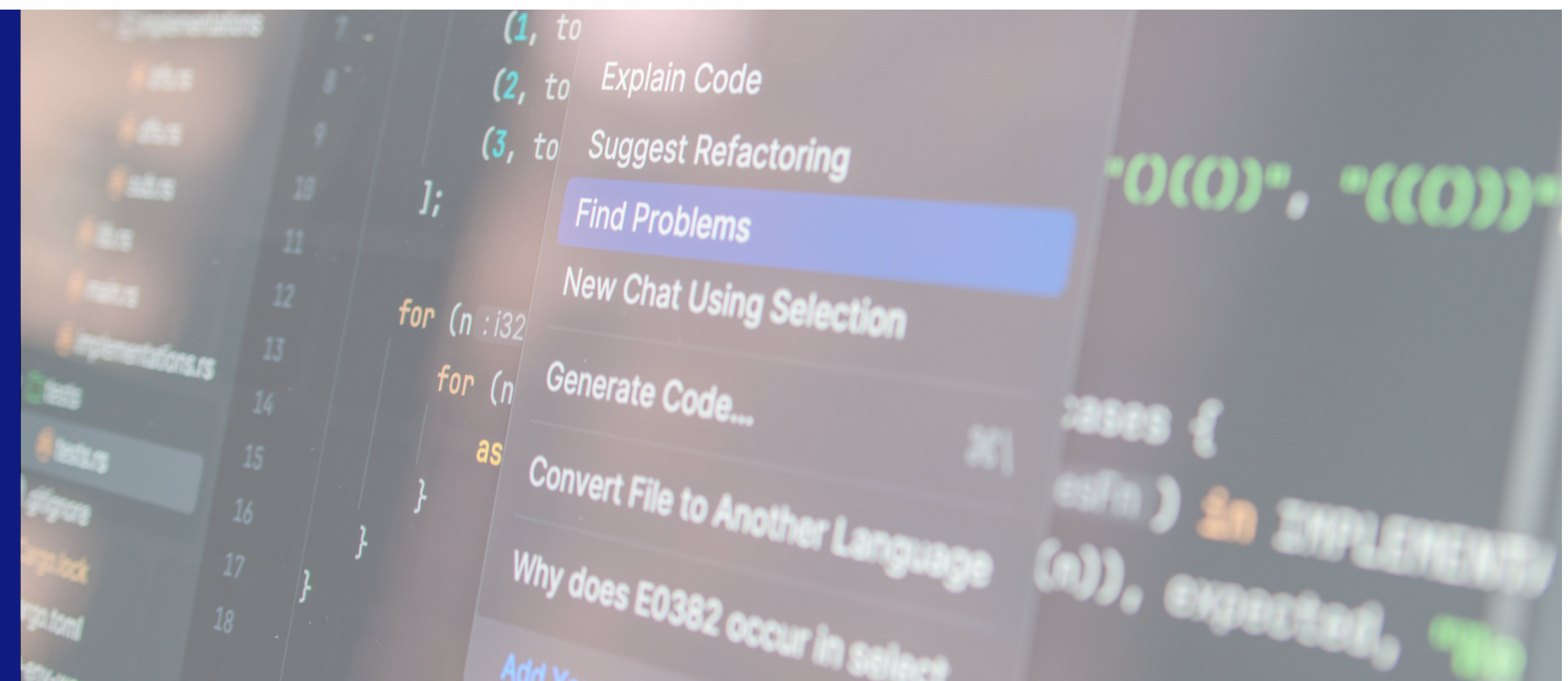
Isolation :

Un service peut être mis à jour ou redémarré sans impacter les autres.

3. Structure du Projet

Configuration Partagée :

Un fichier .env unique à la racine centralise les ports et les secrets (JWT_SECRET, MONGODB_URI) pour assurer la cohérence entre la Gateway et les services.



Middleware Dédiés :

On observe dans l'arborescence que chaque service possède sa propre logique de gestion d'erreurs et de logs, adaptée à son domaine métier.

user-service

```

  user-service
  ├── middleware
  │   ├── JS auth.js
  │   ├── JS errorHandler.js
  │   └── JS logger.js
  ├── models
  │   └── JS User.js
  ├── > node_modules
  ├── routes
  │   └── JS users.js
  ├── {} package-lock.json
  ├── {} package.json
  └── JS server.js
```

story-service

```

  story-service
  ├── middleware
  │   ├── JS auth.js
  │   ├── JS errorHandler.js
  │   └── JS logger.js
  ├── models
  │   └── JS Story.js
  ├── > node_modules
  ├── routes
  │   └── JS stories.js
  ├── {} package-lock.json
  ├── {} package.json
  └── JS server.js
```

post-service

```

  post-service
  ├── middleware
  │   ├── JS auth.js
  │   ├── JS errorHandler.js
  │   └── JS logger.js
  ├── models
  │   └── JS Post.js
  ├── > node_modules
  ├── routes
  │   └── JS posts.js
  ├── {} package-lock.json
  ├── {} package.json
  └── JS server.js
```

comment-service

```

  comment-service
  ├── middleware
  │   ├── JS auth.js
  │   ├── JS errorHandler.js
  │   └── JS logger.js
  ├── models
  │   └── JS Comment.js
  ├── > node_modules
  ├── routes
  │   └── JS comments.js
  ├── {} package-lock.json
  ├── {} package.json
  └── JS server.js
```

IV. ZOOM SUR L'API GATEWAY

1. LES RESPONSABILITÉS DE LA GATEWAY

- **Routage Dynamique (Reverse Proxy) :**

Elle redirige les flux vers les ports internes (3001, 3002, etc.) de manière transparente pour l'utilisateur.

- **Authentification Centralisée :**

Grâce au l'authMiddleware, elle vérifie la validité du Token JWT avant même que la requête n'atteigne les services. Si le token est absent ou invalide, elle bloque l'accès immédiatement (Fail Fast).

- **Sécurité et Protection :**

- Helmet : Sécurise les en-têtes HTTP.
- CORS : Contrôle les accès provenant de domaines tiers.
- Rate Limiting : Protège le système contre les attaques par force brute ou le spam en limitant le nombre de requêtes par IP.

- **Observabilité :**

Elle logue chaque requête entrante (méthode, URL, timestamp) pour faciliter le débogage.

2.LE PATTERN EN ACTION

- La chaîne de Middlewares

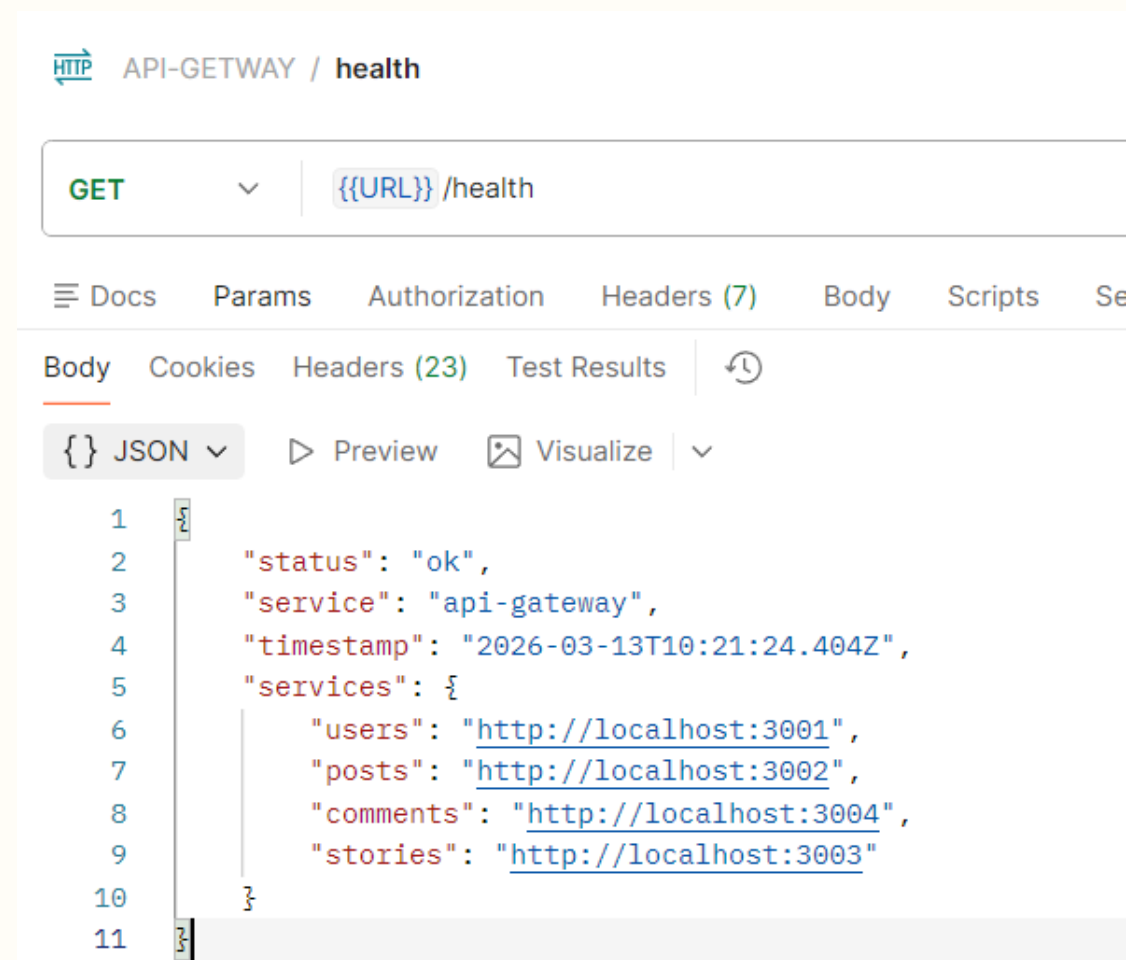
```
// Montage des protections globales
applySecurityMiddleware(app); //
Protection réseau
app.use(authMiddleware);      //
Vérification identité
app.use(logger);              //
Traçabilité
```

- La configuration d'un Proxy

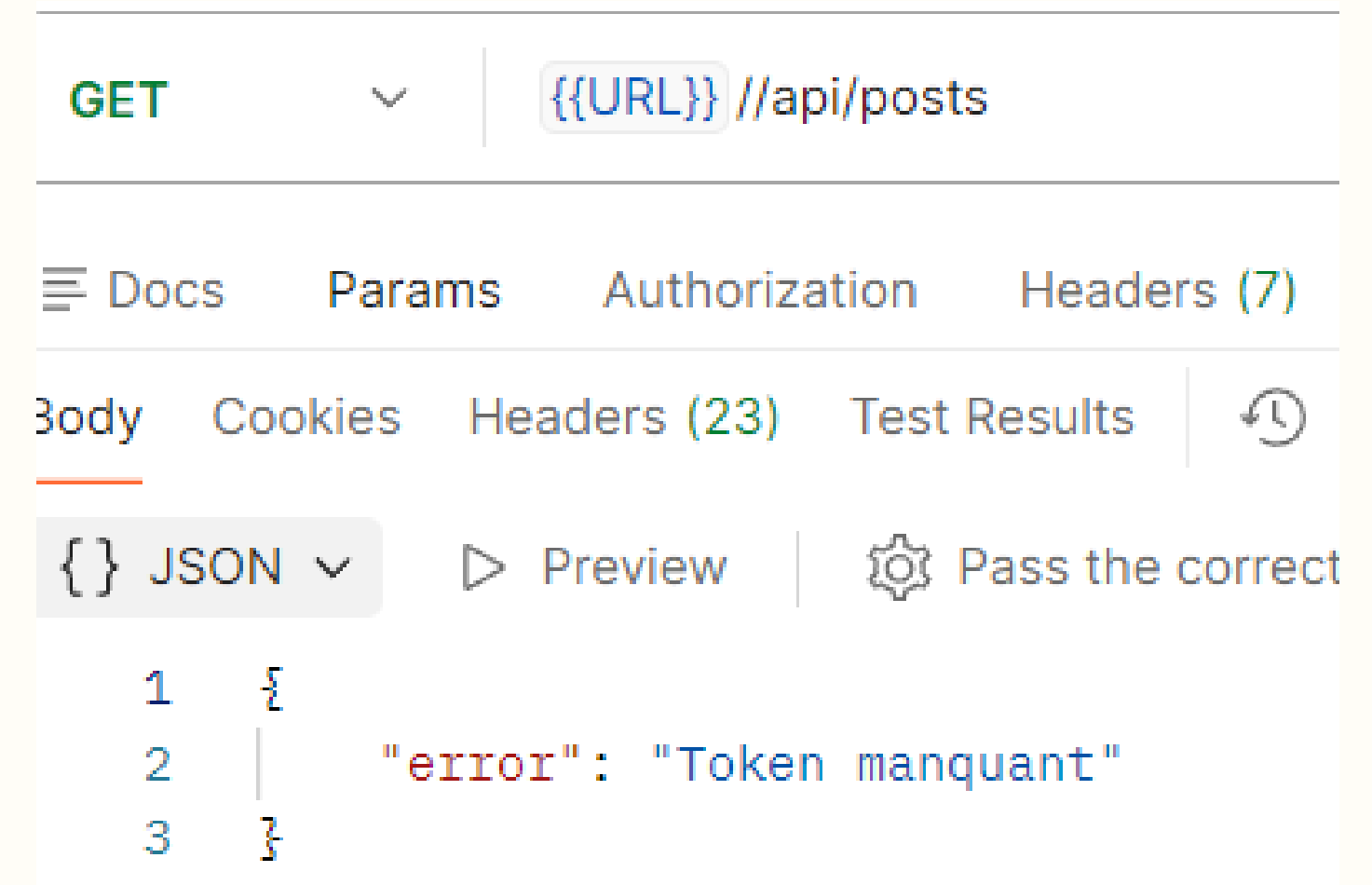
```
app.use("/api/users",
  createProxyMiddleware({
    target: `http://localhost:3001`,
    changeOrigin: true,
    onProxyReq: fixRequestBody, //
    // Assure le transfert du JSON
    onError: (err, req, res) => { ... }
  }));
```

2.LE PATTERN EN ACTION

- Le Health Check Global



- Tentative d'accès sans Token



V. PRÉSENTATION DES SERVICES MÉTIERS

USER-SERVICE

1. MODÈLE DE DONNÉES (USER)

- **Informations clés : username (unique), email (unique), password (haché), bio, avatar.**
- **Réseau social : Liste de références (ObjectID) vers les followers et le following.**
- **Fonctionnalités intégrées : Masquage automatique du mot de passe dans les réponses JSON, calcul virtuel du nombre d'abonnés.**

USER-SERVICE

2. POINTS D'ENTRÉE API (/API/USERS)

Authentication :

- **POST /register** : Crée un compte, hache le mot de passe, retourne un token JWT.
- **POST /login** : Vérifie les identifiants, retourne un token JWT.

Profils :

- **GET /:id** : Récupère le profil détaillé d'un utilisateur, en incluant les noms et avatars des abonnés/abonnements (jointures Mongoose).

Interactions Sociales (Routes protégées) :

- **POST /:id/follow** : Permet de suivre un autre utilisateur, mettant à jour les deux profils.
- **DELETE /:id/unfollow** : Arrête de suivre un utilisateur, mettant à jour les deux profils.

USER-SERVICE

3. CONFIGURATION DU SERVEUR (SERVER.JS)

Authentication :

- **POST /register** : Crée un compte, hache le mot de passe, retourne un token JWT.
- **POST /login** : Vérifie les identifiants, retourne un token JWT.

Profils :

- **GET /:id** : Récupère le profil détaillé d'un utilisateur, en incluant les noms et avatars des abonnés/abonnements (jointures Mongoose).

Interactions Sociales (Routes protégées) :

- **POST /:id/follow** : Permet de suivre un autre utilisateur, mettant à jour les deux profils.
- **DELETE /:id/unfollow** : Arrête de suivre un utilisateur, mettant à jour les deux profils.

USER-SERVICE

The screenshot displays a REST client interface with a POST request to `/api/users/register`. The request body is a JSON object with the following fields:

```
{
  "username": "asma",
  "email": "asma@gmail.com",
  "password": "asma"
}
```

The response status is **201 Created** with a response time of 376 ms. The response body is a JSON object:

```
{
  "message": "Utilisateur créé avec succès",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQ0IjI0WlZlZWY4MmQ5OGIyNDhmZDI3N2U4ImV4cCI6MTc3NDAwNDczOH0uBjUJ3SBHjNq-U2ybcaQ0",
  "user": {
    "id": "69b3ef82d98b248fd277e850",
    "username": "asma",
    "email": "asma@gmail.com"
  }
}
```

`{{URL}}/API/USERS/REGISTER`

The screenshot shows the Postman interface for a POST request to `/api/users/login`. The request body is in JSON format, containing `email` and `password` fields. The response is a 200 OK status with a response time of 943 ms and a size of 1.27 KB. The response body is in JSON format, containing `message`, `token`, and `user` fields. The `token` field is highlighted in blue, and the `user` field is highlighted in yellow.

```
{
  "email": "asma@gmail.com",
  "password": "asma"
}
```

Body 200 OK • 943 ms • 1.27 KB

{ } JSON Preview Visualize

```
{
  "message": "Connexion réussie",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySwQ1OiI2OWIzZWY4MmQ5OGIyNDhmZDI3N2U4NTIyImV4cCI6MTc3NDAwNTAzNX0.18djhlUXTRoYFABm651ZpKl",
  "user": {
    "id": "69b3ef82d98b248fd277e850",
    "username": "asma",
    "email": "asma@gmail.com"
  }
}
```

{{URL}}/API/USERS/LOGIN

The screenshot shows the Swagger UI interface for a REST API. At the top, there's a navigation bar with "HTTP" and "user / login". Below this, the request method is "GET" and the URL is "/api/users/69b3ef82d98b248fd277e850". The "Send" button is visible. The "Auth" tab is selected, showing "Bearer Token" as the authentication type. The response status is "200 OK" with a response time of "93 ms" and a size of "1.22 KB". The response body is a JSON object representing a user:

```
{
  "_id": "69b3ef82d98b248fd277e850",
  "username": "asma",
  "email": "asma@gmail.com",
  "bio": "",
  "avatar": "https://via.placeholder.com/150",
  "followers": [],
  "following": [],
  "createdAt": "2026-03-13T11:05:38.769Z",
  "updatedAt": "2026-03-13T11:05:38.769Z",
  "__v": 0
}
```

{{URL}}/API/USERS/ID_USER

USER-SERVICE

HTTP

user / follow

Save

Share

POST

{{URL}}/api/users/69b3ef82d98b248fd277e850/follow

Send

Docs

Params

Auth

Headers (10)

Body

Scripts

Settings

Cookies

Auth Type

Bearer Token

Token

.....

Body

200 OK

312 ms

1.02 KB

Save Response

JSON

Preview

Visualize

1

2

3

{

"message": "Utilisateur suivi avec succès"

}

HTTP

user / unfollow

Save

Share

DELETE

{{URL}}/api/users/69b3ef82d98b248fd277e850/unfollow

Send

Docs

Params

Auth

Headers (10)

Body

Scripts

Settings

Cookies

Auth Type

Bearer Token

Token

.....

Body

200 OK

367 ms

1.01 KB

Save Response

JSON

Preview

Visualize

1

2

3

{

"message": "Utilisateur non suivi"

}

POST
{{URL}}/API/USERS/ID_USER/FOLLOW

DELETE
{{URL}}/API/USERS/ID_USER/UNFOLLOW

POST-SERVICE

1. MODÈLE DE DONNÉES (POST)

- **Informations clés :** `userId` (référence), `content` (obligatoire, max 2000 car.), `images` (tableau d'URLs validées), `visibility` (public, friends, private).
- **Interactions :** `likesCount` (dénormalisé), `likedBy` (tableau d'IDs), `commentsCount`, `sharesCount`.
- **Optimisation :** Index sur `userId`, `visibility`, `likedBy`.
- **Logique intégrée :** Synchronisation automatique de `likesCount` avant sauvegarde. Méthodes pour gérer les likes de manière atomique par utilisateur (`addLike`, `removeLike`, `hasLiked`).

POST-SERVICE

2. POINTS D'ENTRÉE API (/API/POSTS)

Flux de publications :

- GET / : Récupère le feed public avec pagination et tri antéchronologique.
- GET /user/:userId : Récupère toutes les publications d'un utilisateur spécifique.

Gestion individuelle :

- POST / (auth requis) : Crée une nouvelle publication pour l'utilisateur connecté.
- GET /:id : Récupère une publication spécifique.
- PUT /:id (auth requis) : Le propriétaire modifie sa publication.
- DELETE /:id (auth requis) : Le propriétaire supprime sa publication.

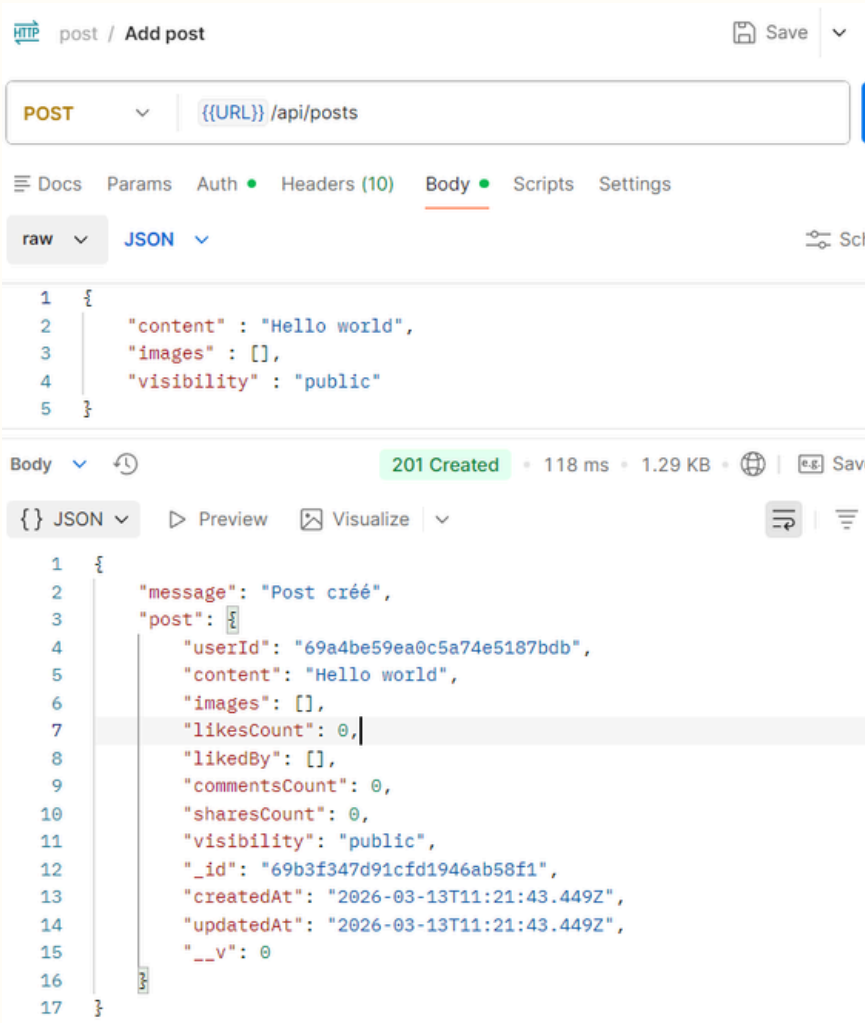
Interactions :

- POST /:id/like (auth requis) : L'utilisateur connecté like la publication (une fois).
- POST /:id/unlike (auth requis) : L'utilisateur connecté retire son like.

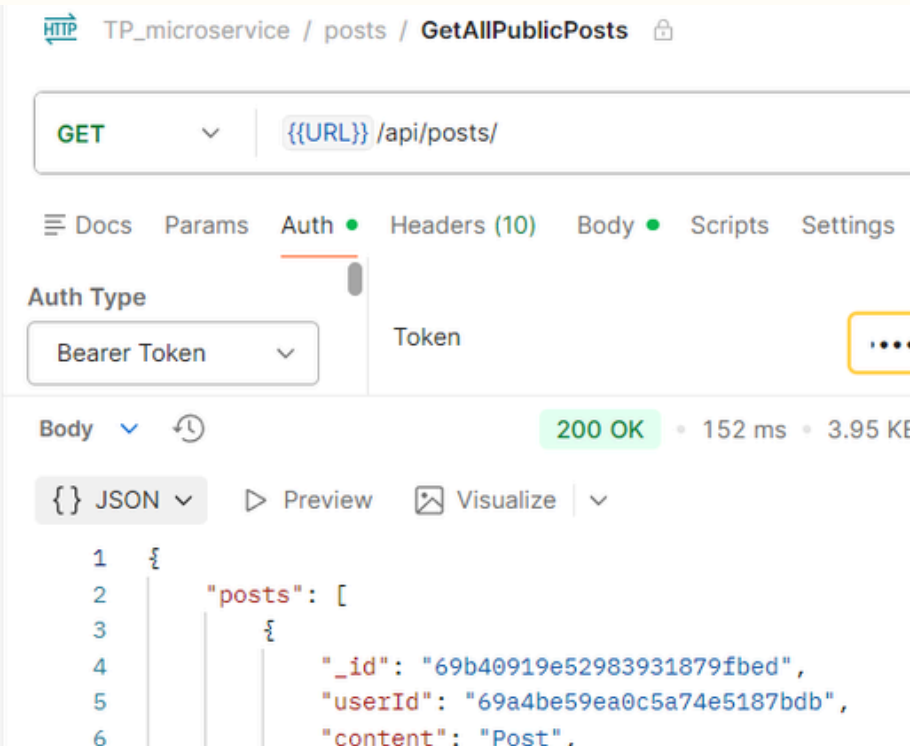
3. CONFIGURATION DU SERVEUR (SERVER.JS)

La même logique que User Service.

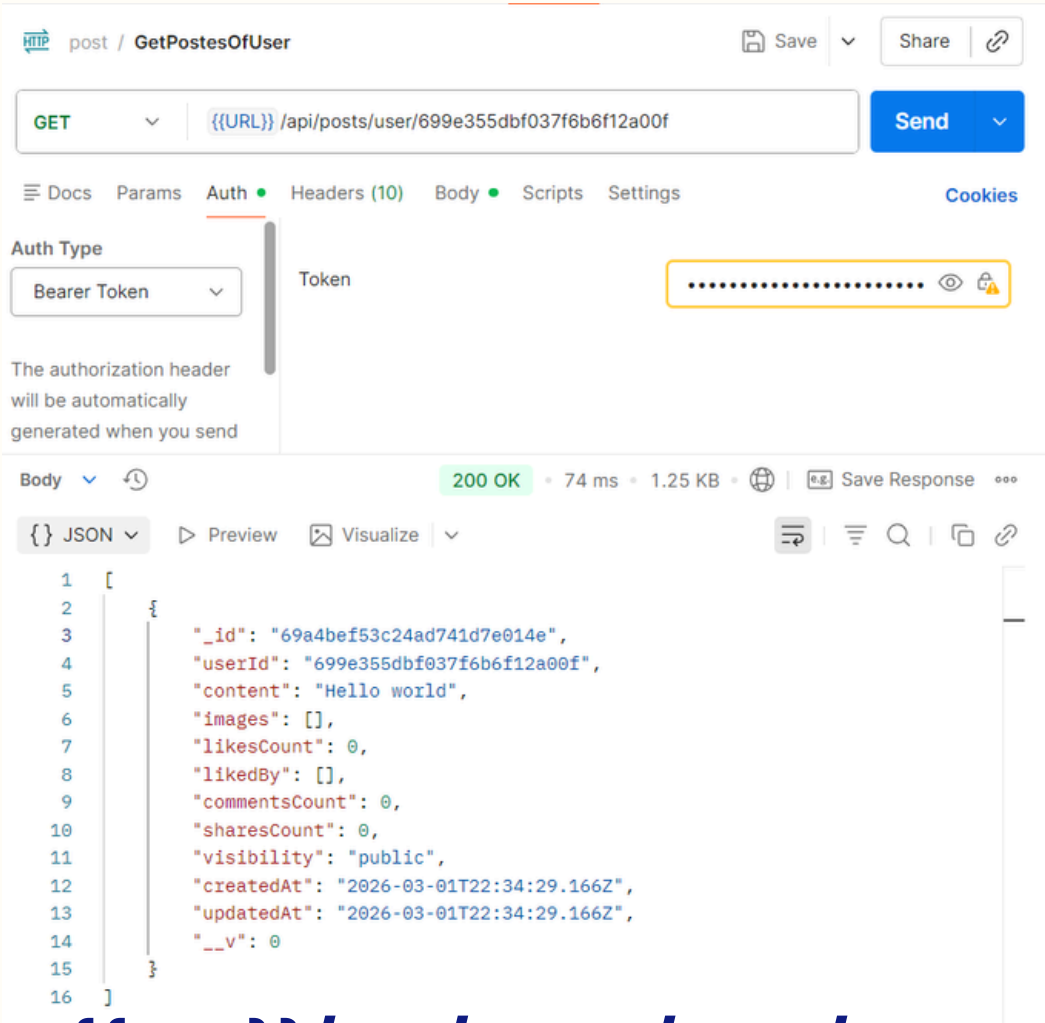
POST-SERVICE



POST {{URL}}/api/posts

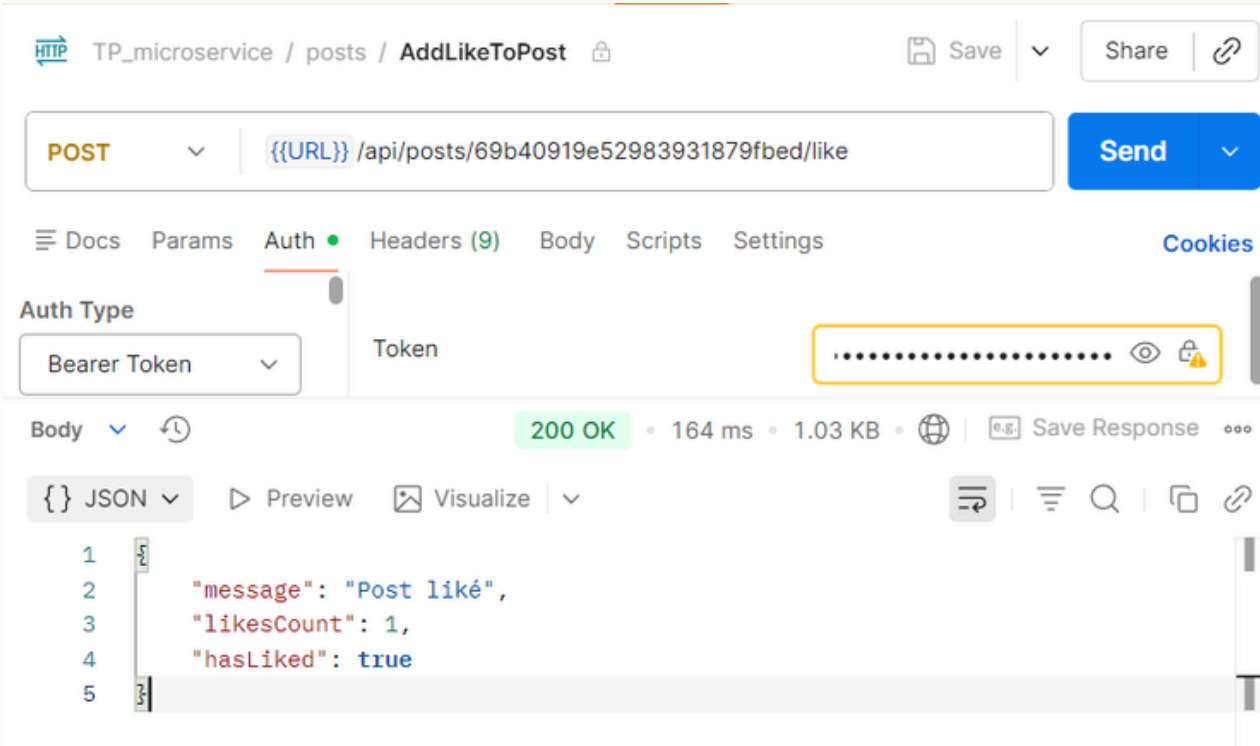


GET {{URL}}/api/posts (public)

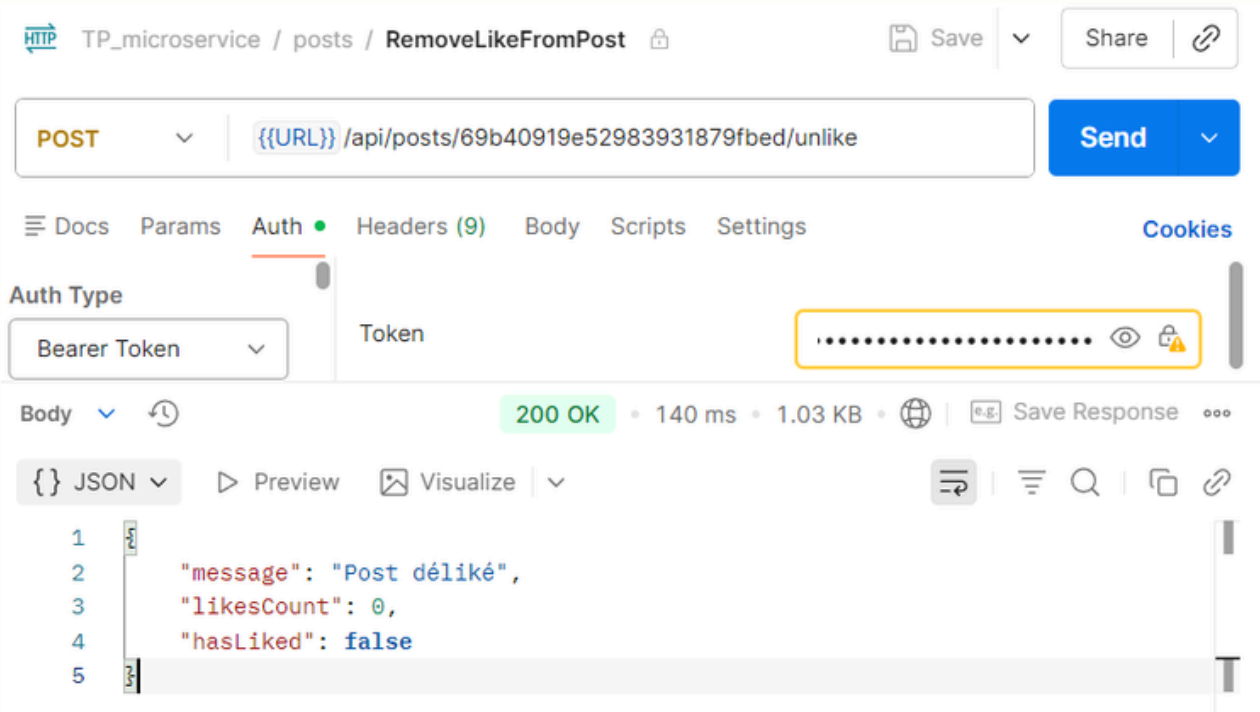


GET {{URL}}/api/posts/user/:UserId

POST-SERVICE



POST {{URL}}/api/posts/PostId/like



DELETE {{URL}}/api/posts/PostId/unlike

STORY-SERVICE

1. MODÈLE DE DONNÉES (STORY)

- Informations clés : **userId** (référence), **content** (obligatoire), **mediaUrl** & **mediaType** (optionnels), **visibility**.
- Temporarisation & Vues : **expiresAt** (obligatoire), **viewers** (tableau d'IDs), **viewsCount** (dénormalisé).
- Optimisation : Index sur **userId**, **visibility**, et un index TTL sur **expiresAt** pour la suppression automatique.
- Logique intégrée : Nettoyage automatique du tableau **viewers** (unicité, exclusion du créateur) et mise à jour de **viewsCount** avant sauvegarde. Méthode pour ajouter une vue.

STORY-SERVICE

2. POINTS D'ENTRÉE API (/API/STORIES)

Flux (uniquement les stories non expirées) :

- **GET / :** Récupère le feed public, paginé, filtrable par userId.
- **GET /user/:userId :** Récupère les stories actives d'un utilisateur spécifique.

Gestion individuelle :

- **POST / (auth requis) :** Crée une story. Calcule expiresAt (entre 1h et 48h, défaut 24h).
- **GET /:id :** Récupère une story active spécifique.
- **DELETE /:id (auth requis) :** Le propriétaire supprime sa story.

Interactions :

- **POST /:id/view (auth requis) :** Enregistre la vue de l'utilisateur connecté sur une story active.

3. CONFIGURATION DU SERVEUR (SERVER.JS)

La même logique que User Service.

STORY-SERVICE

TP_microservice / Story / AddStory

POST {{URL}} /api/stories

Auth Type: Bearer Token

201 Created • 148 ms • 1.53 KB

```
1 {
2   "message": "Story créée",
3   "story": {
4     "userId": "69a4be59ea0c5a74e5187bdb",
5     "content": "Hello",
6     "mediaUrl": "https://media.istockphoto.com/id/1550071750/fr/photo/larbre-%C3%A0-th%C3%A9-vert-laisse-camellia-sinensis-dans-la-lumi%C3%A8re-du-soleil-de-la-ferme-biologique.jpg?s=612x612&w=0&k=20&c=yb7sbgw8i5o_8c3cHbx7BpkFhPGFiICy_wPs2rFHS_o=",
7     "mediaType": "image",
8     "visibility": "public",
9     "viewers": [],
10    "viewsCount": 0,
11    "expiresAt": "2026-03-14T13:01:03.623Z",
12    "_id": "69b40a8f489d89c43d19dbfa",
13    "createdAt": "2026-03-13T13:01:03.636Z",
14    "updatedAt": "2026-03-13T13:01:03.636Z",
15    "__v": 0
16  }
17 }
```

POST {{URL}}/api/stories

TP_microservice / Story / GetAllPublicStory

GET {{URL}} /api/stories

Auth Type: Bearer Token

200 OK • 155 ms • 1.56 KB

```
1 {
2   "stories": [
3     {
4       "_id": "69b40a8f489d89c43d19dbfa",
5       "userId": "69a4be59ea0c5a74e5187bdb",
6       "content": "Hello",
7       "mediaUrl": "https://media.istockphoto.com/id/1550071750/fr/photo/larbre-%C3%A0-th%C3%A9-vert-laisse-camellia-sinensis-dans-la-lumi%C3%A8re-du-soleil-de-la-ferme-biologique.jpg?s=612x612&w=0&k=20&c=yb7sbgw8i5o_8c3cHbx7BpkFhPGFiICy_wPs2rFHS_o=",
8       "mediaType": "image",
9       "visibility": "public",
10      "viewers": [],
11      "viewsCount": 0,
12      "expiresAt": "2026-03-14T13:01:03.623Z"
13    }
14  ]
15 }
```

GET {{URL}}/api/stories

TP_microservice / Story / GetStoryOfUser

GET {{URL}} /api/stories/user/69a4be59ea0c5a74e5187bdb

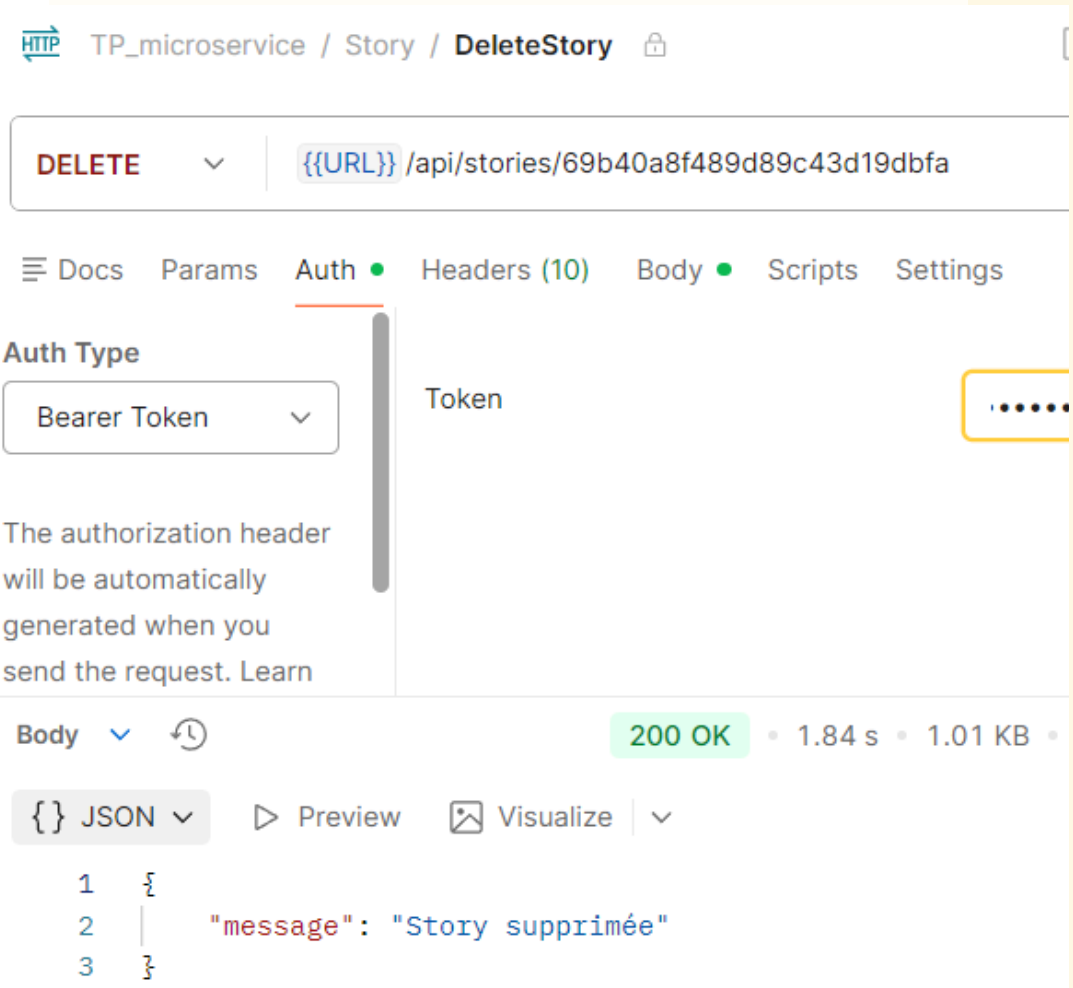
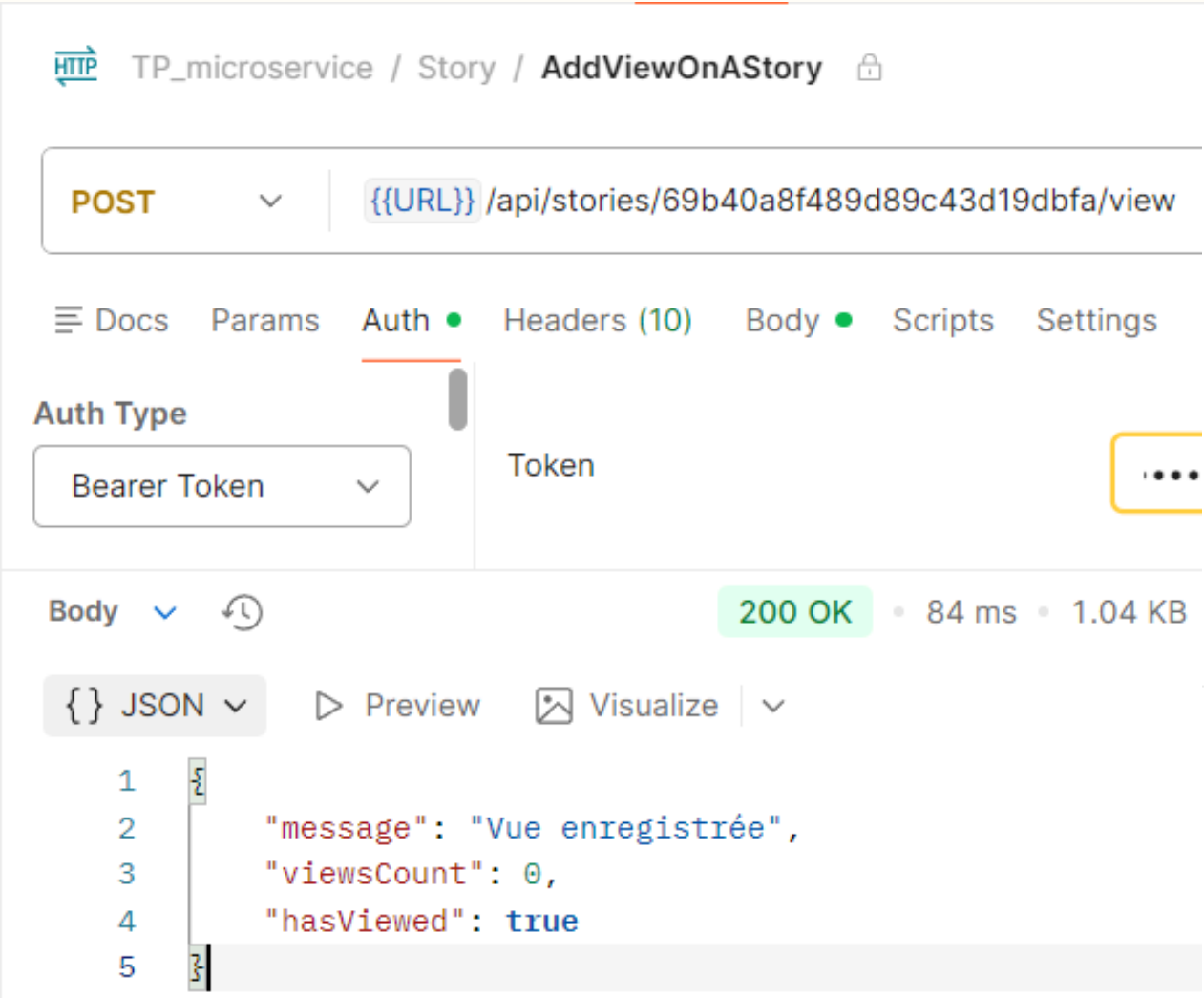
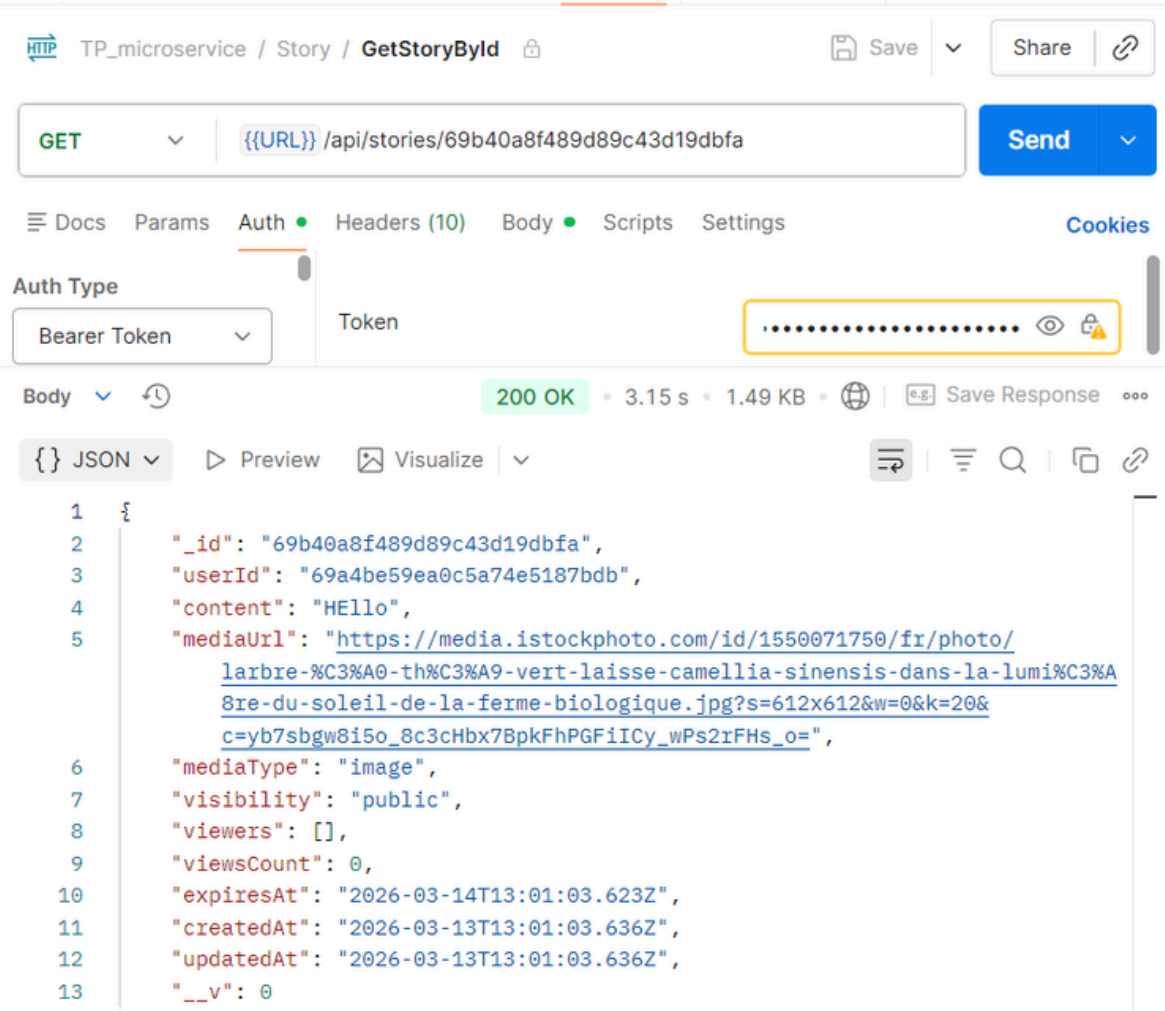
Auth Type: Bearer Token

200 OK • 70 ms • 1.49 KB

```
1 {
2   "_id": "69b40a8f489d89c43d19dbfa",
3   "userId": "69a4be59ea0c5a74e5187bdb",
4   "content": "Hello",
5   "mediaUrl": "https://media.istockphoto.com/id/1550071750/fr/photo/larbre-%C3%A0-th%C3%A9-vert-laisse-camellia-sinensis-dans-la-lumi%C3%A8re-du-soleil-de-la-ferme-biologique.jpg?s=612x612&w=0&k=20&c=yb7sbgw8i5o_8c3cHbx7BpkFhPGFiICy_wPs2rFHS_o=",
6   "mediaType": "image",
7   "visibility": "public",
8   "viewers": [],
9   "viewsCount": 0,
10  "expiresAt": "2026-03-14T13:01:03.623Z",
11  "createdAt": "2026-03-13T13:01:03.636Z",
12  "updatedAt": "2026-03-13T13:01:03.636Z"
13 }
```

GET {{URL}}/api/stories/user/UserId

STORY-SERVICE



POST {{URL}}/api/stories/:StoryId POST {{URL}}/api/stories/:StoryId/view DELETE {{URL}}/api/stories/:StoryId

COMMENT-SERVICE

1. MODÈLE DE DONNÉES (STORY)

- Informations clés : postId (référence), userId (référence), content (obligatoire, max 1000 car.).
- Structure imbriquée : parentCommentId (référence récursive vers un autre Comment) permettant de gérer des réponses à des commentaires.
- Optimisation : Index composés sur postId (avec tri décroissant) et parentCommentId (avec tri croissant) pour accélérer la récupération des fils de discussion.

COMMENT-SERVICE

2. POINTS D'ENTRÉE API (/API/COMMENTS)

Création & Lecture :

- **POST / (auth requis) :** Crée un commentaire principal ou une réponse si `parentCommentId` est fourni.
- **GET / :** Récupère les commentaires principaux (`parentCommentId: null`) d'un post spécifique (via query param), paginés et triés par date décroissante.
- **GET /post/:postId :** Récupère tous les commentaires principaux d'un post spécifique.
- **GET /:id :** Récupère un commentaire spécifique par son ID.
- **GET /:id/replies :** Récupère toutes les réponses associées à un commentaire parent, triées chronologiquement.

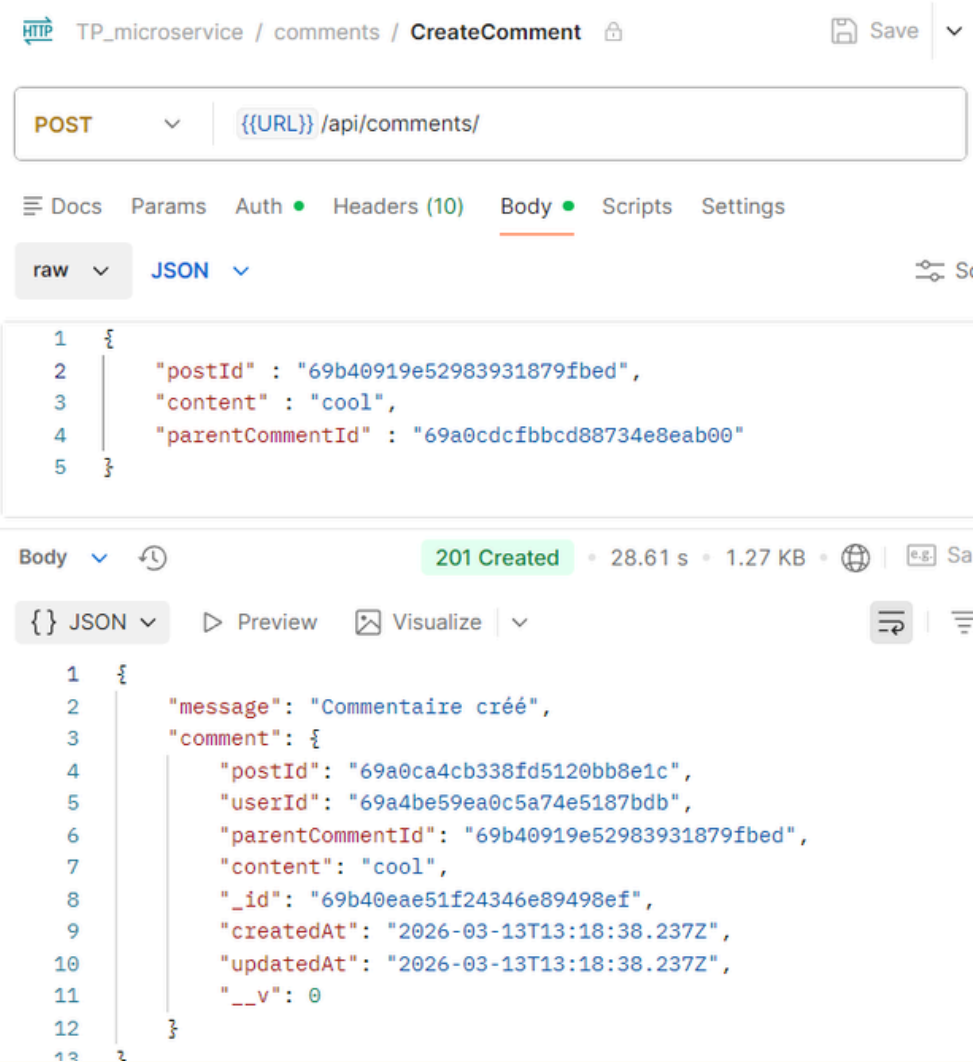
Gestion individuelle (auth requis) :

- **PUT /:id :** Le propriétaire modifie le contenu de son commentaire.
- **DELETE /:id :** Le propriétaire supprime son commentaire. Important : Cette action entraîne la suppression en cascade de toutes les réponses associées via `Comment.deleteMany`.

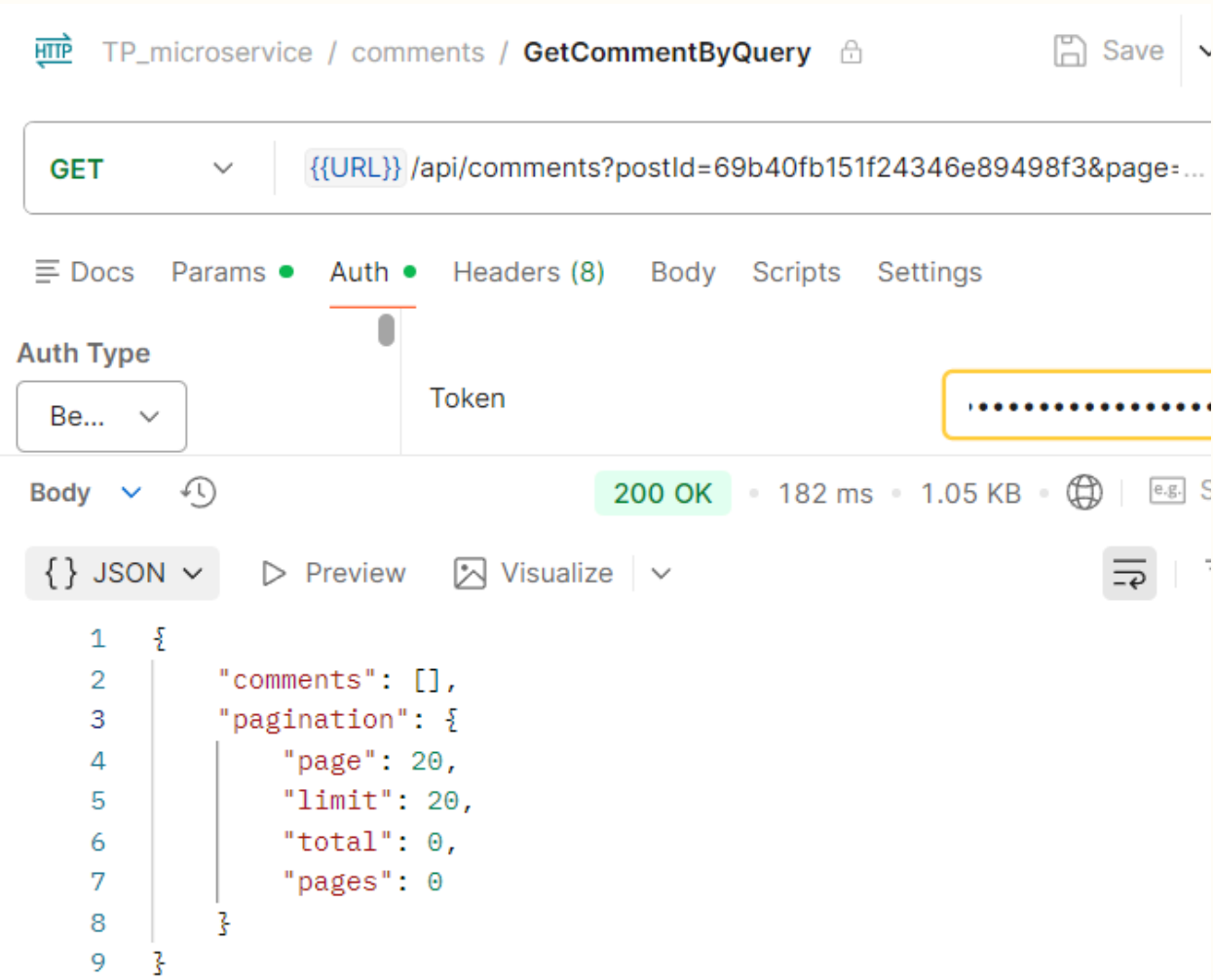
3. CONFIGURATION DU SERVEUR (SERVER.JS)

La même logique que User Service.

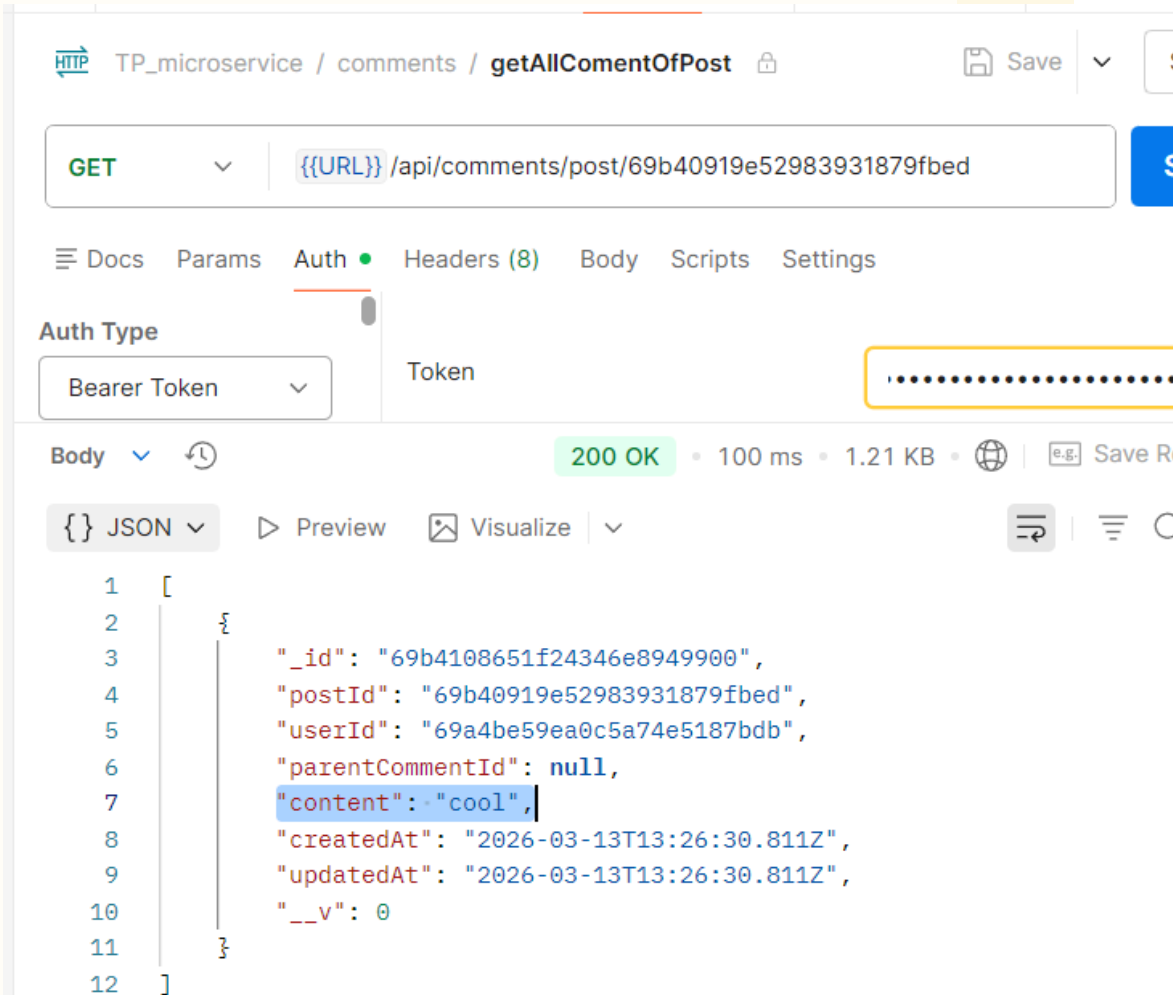
COMMENT-SERVICE



{{URL}}/api/comments



{{URL}}/api/comments/post/:postId



{{URL}}/api/comments/post/:postId

COMMENT-SERVICE

TP_microservice / comments / GetCommentById

GET {{URL}}/api/comments/69b4108651f24346e8949900

Auth Type: Bearer Token

Token:

Body: 200 OK • 398 ms • 1.2 KB

```
1 {
2   "_id": "69b4108651f24346e8949900",
3   "postId": "69b40919e52983931879fbcd",
4   "userId": "69a4be59ea0c5a74e5187bdb",
5   "parentCommentId": null,
6   "content": "cool",
7   "createdAt": "2026-03-13T13:26:30.811Z",
8   "updatedAt": "2026-03-13T13:26:30.811Z",
9   "__v": 0
10 }
```

TP_microservice / comments / DeleteComment

DELETE {{URL}}/api/comments/69b4108651f24346e8949900

Auth Type: Bearer Token

Token:

Body: 200 OK • 241 ms • 1.01 KB

```
1 {
2   "message": "Commentaire supprimé"
3 }
```

{{URL}}/api/comments/:CommentId

GET {{URL}}/api/comments/post/69b40919e52983931879fbcd

Auth Type: Bearer Token

Token:

Body: 200 OK • 154 ms • 1000 B

```
1 []
```

COMMENT-SERVICE

TP_microservice / comments / **GetCommentById**

GET `{{URL}}/api/comments/69b4108651f24346e8949900` Send

Docs Params Auth Headers (8) Body Scripts Settings

Auth Type: Bearer Token

Token:

Body: 200 OK • 398 ms • 1.2 KB

```
1 {
2   "_id": "69b4108651f24346e8949900",
3   "postId": "69b40919e52983931879fbcd",
4   "userId": "69a4be59ea0c5a74e5187bdb",
5   "parentCommentId": null,
6   "content": "cool",
7   "createdAt": "2026-03-13T13:26:30.811Z",
8   "updatedAt": "2026-03-13T13:26:30.811Z",
9   "__v": 0
10 }
```

`{{URL}}/api/comments/:CommentId`

TP_microservice / comments / **DeleteComment**

DELETE `{{URL}}/api/comments/69b4108651f24346e8949900` Send

Docs Params Auth Headers (8) Body Scripts Settings

Auth Type: Bearer Token

Token:

Body: 200 OK • 241 ms • 1.01 KB

```
1 {
2   "message": "Commentaire supprimé"
3 }
```

GET `{{URL}}/api/comments/post/69b40919e52983931879fbcd` Send

Docs Params Auth Headers (8) Body Scripts Settings Cookies

Auth Type: Bearer Token

Token:

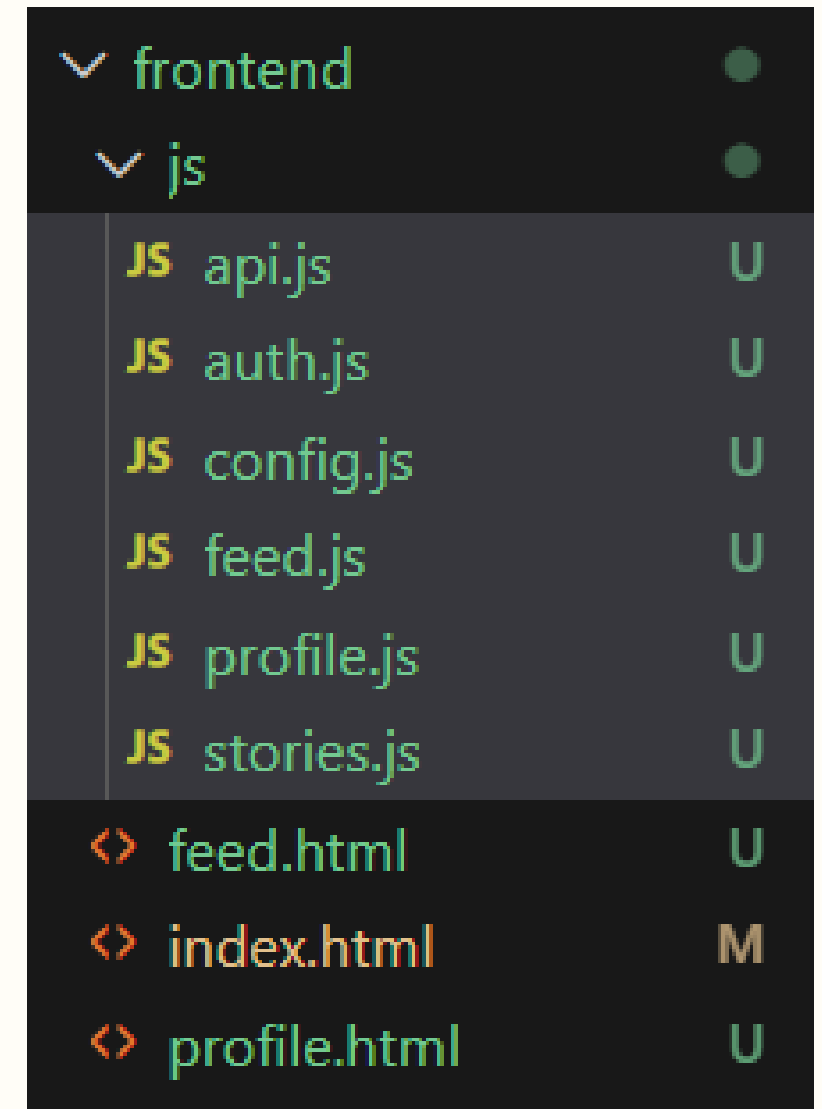
Body: 200 OK • 154 ms • 1000 B

```
1 []
```

VI. INTÉGRATION DE L'INTERFACE UTILISATEUR (FRONTEND)

VALIDATION DES MICROSERVICES VIA UN FRONTEND LÉGER

- Approche minimaliste : HTML, Vanilla JS et Tailwind via CDN pour se concentrer sur la communication avec l'API Gateway.
- Architecture modulaire : Code JS découpé par fonctionnalité pour refléter la structure des microservices backend.
- Validation des flux : Interface permettant de tester concrètement le parcours complet (authentification, publications, stories, profil).
- Rôle de preuve de concept : Démontre l'efficacité de la Gateway dans le routage et la validation des tokens JWT.



▼ frontend	●
▼ js	●
JS api.js	U
JS auth.js	U
JS config.js	U
JS feed.js	U
JS profile.js	U
JS stories.js	U
<> feed.html	U
<> index.html	M
<> profile.html	U

Mini Social

Register

Login

PAGE LOGIN/REGISTER

← → ↻ ⓘ 127.0.0.1:5500/frontend/feed.html 🔑 🔍 ☆

Feed

Logout

Profile

Post

Stories

Add Story

FIL D'ACTUALITÉ

← → ↻ ⓘ 127.0.0.1:5500/frontend/profile.html

Profile

Posts:

PROFILE

37

Stories

first story |

Add Story

hiii

hiii

hiii

AJOUT DE STORY

first post

 1

write comment...

comment

INTERACTION SUR LE POST

first post

 0

write comment...

comment

INTERACTION SUR LE POST

first post

Post

AJOUT DE POST

VII. SYNTHÈSE FINALE : ARCHITECTURE & PERSPECTIVES

ARCHITECTURE & PERSPECTIVES

- **Patterns Transverses :**

**L'authentification repose sur un JWT partagé, centralisé au niveau de l'API Gateway .
L'isolation des données est assurée logiquement par collection pour chaque microservice ("Database per Service"), même avec une URI commune .**

- **Conclusion & Évolutions :**

**Cette architecture offre une grande facilité de déploiement et une robustesse accrue.
Les prochaines étapes incluent la dockerisation, la mise en place du Service Discovery (Consul/Eureka), et l'intégration d'un Message Broker (RabbitMQ) pour une communication asynchrone .**

MERCI POUR VOTRE ATTENTION